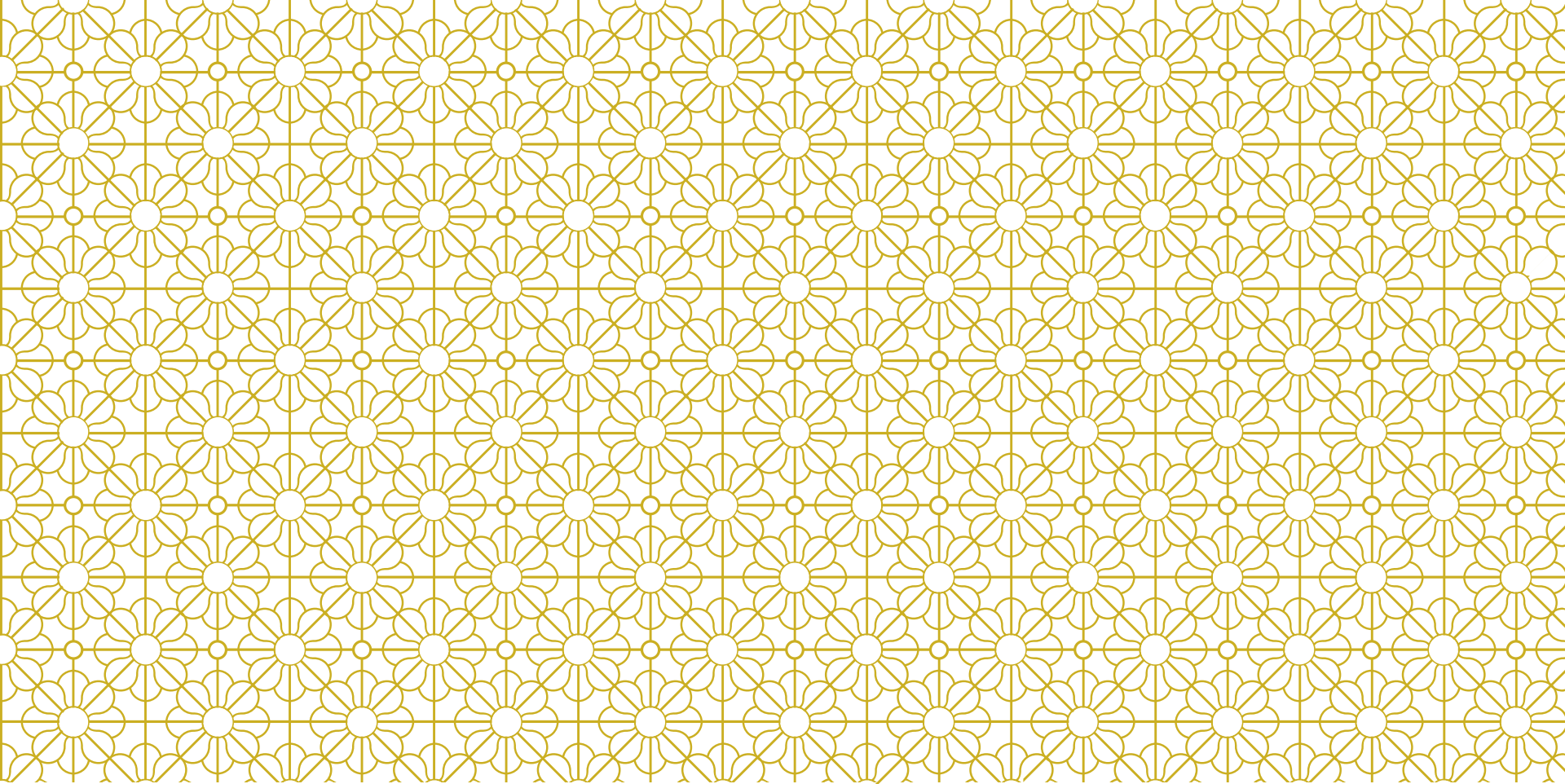
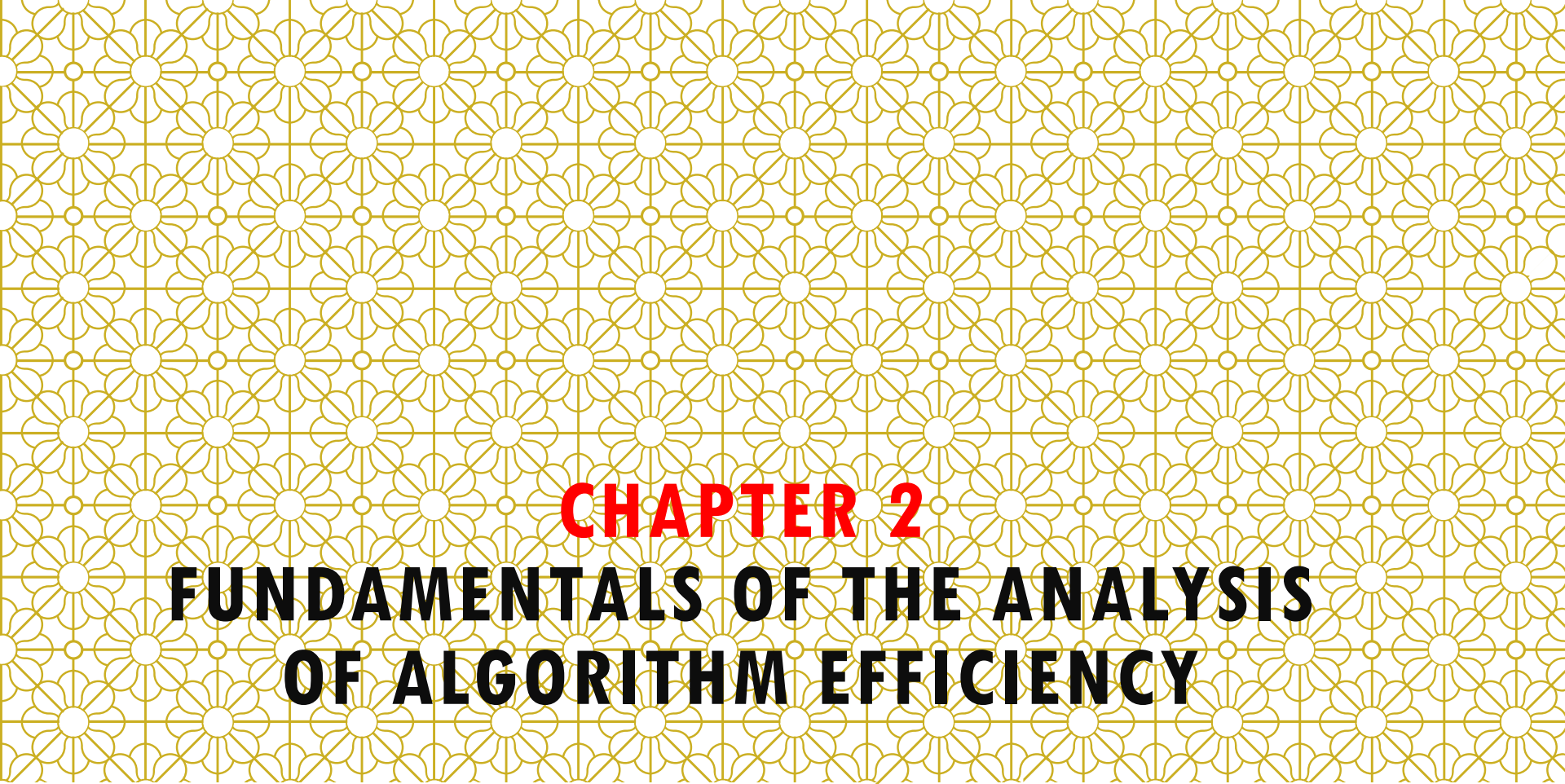




ALGORITHMS ANALYSIS AND DESIGN

By : Dr. Basem
Alijla



CHAPTER 2

FUNDAMENTALS OF THE ANALYSIS OF ALGORITHM EFFICIENCY

OUTLINE

The Analysis Framework

Asymptotic Notations & Basic Efficiency Classes

Analysis of Non-recursive Algorithms

Analysis of Recursive Algorithms

EXAMPLE 1: RECURSIVE EVALUATION OF N!

Definition:

$$n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n \text{ for } n > 0 \text{ and } 0! = 1$$

Recursive definition of $n!$:

$$F(n) = F(n-1) \cdot n \text{ for } n \geq 1 \text{ and } F(0) = 1$$

ALGORITHM $F(n)$

```
//Computes  $n!$  recursively
//Input: A nonnegative integer  $n$ 
//Output: The value of  $n!$ 
if  $n = 0$  return 1
else return  $F(n - 1) * n$ 
```

$$\begin{aligned} 5! &= 4! \times 5 \\ &= 3! \times 4 \\ &= 2! \times 3 \\ &= 1! \times 2 \\ &= 0! \times 1 \\ &= 1 \end{aligned}$$

ANALYSIS PLAN OF RECURSIVE ALGORITHMS

- Decide on a parameter indicating an input's size.
- Identify the algorithm's basic operation.
- Check whether the number of times the basic op. is executed may vary on different inputs of the same size. (If it may, the worst, average, and best cases must be investigated separately.)

ANALYSIS PLAN OF RECURSIVE ALGORITHMS

- Set up a recurrence relation with an appropriate initial condition expressing the number of times the basic op. is executed.
- Solve the recurrence (or, at the very least, establish its solution's order of growth) by backward substitutions or another method.

EXAMPLE 1: RECURSIVE EVALUATION OF N!

Definition:

$$n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n \text{ for } n > 0 \text{ and } 0! = 1$$

Recursive definition of $n!$:

$$F(n) = F(n-1) \cdot n \text{ for } n \geq 1 \text{ and } F(0) = 1$$

ALGORITHM $F(n)$

```
//Computes  $n!$  recursively  
//Input: A nonnegative integer  $n$   
//Output: The value of  $n!$   
if  $n = 0$  return 1  
else return  $F(n - 1) * n$ 
```

EXAMPLE 1: RECURSIVE EVALUATION OF N!

➤ **Size:** N

➤ **Basic operation:**

✓ **Multiplication** , Number of multiplications **M(n)**

✓ Number of calls made by the algorithm or Comparing $n == 0$

➤ **Recurrence relation:** $F(n) = F(n - 1) \times n$ for $n \geq 1$

$$M(n) = M(n-1) + 1$$

To compute
 $F(n-1)$

To multiply
 $F(n-1)$ by n

EXAMPLE 1: RECURSIVE EVALUATION OF N!

➤ Initial Condition or Stop Condition, **$M(0) = 0$**

$$F(n) = F(n-1) \cdot n \text{ for } n > 0$$

$$F(0) = 1$$

Solve the recurrence relation by method of **Backward substitutions**

BACKWARD SUBSTITUTIONS

$$M(n) = M(n - 1) + 1$$

$$= [M(n-2) + 1] + 1 = M(n-2) + 2$$

$$= [M(n-3) + 1] + 2 = M(n-3) + 3$$

.....

$$= M(n - n) + n$$

$$= M(0) + n$$

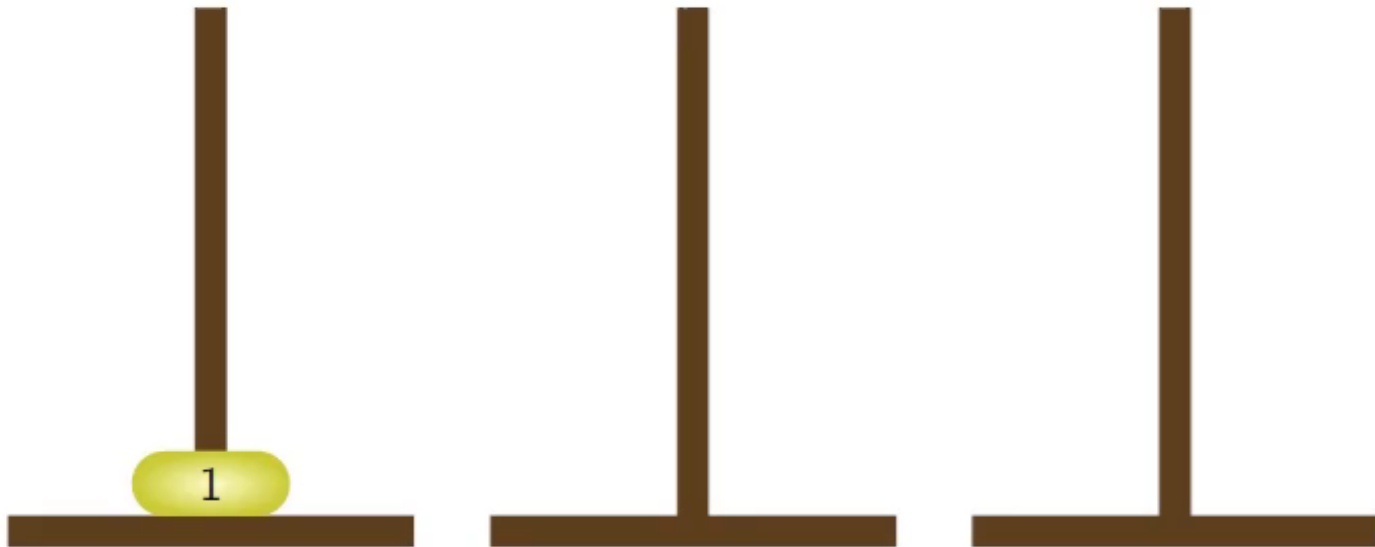
$$= 0 + n$$

$$= n$$

$$O(n)$$

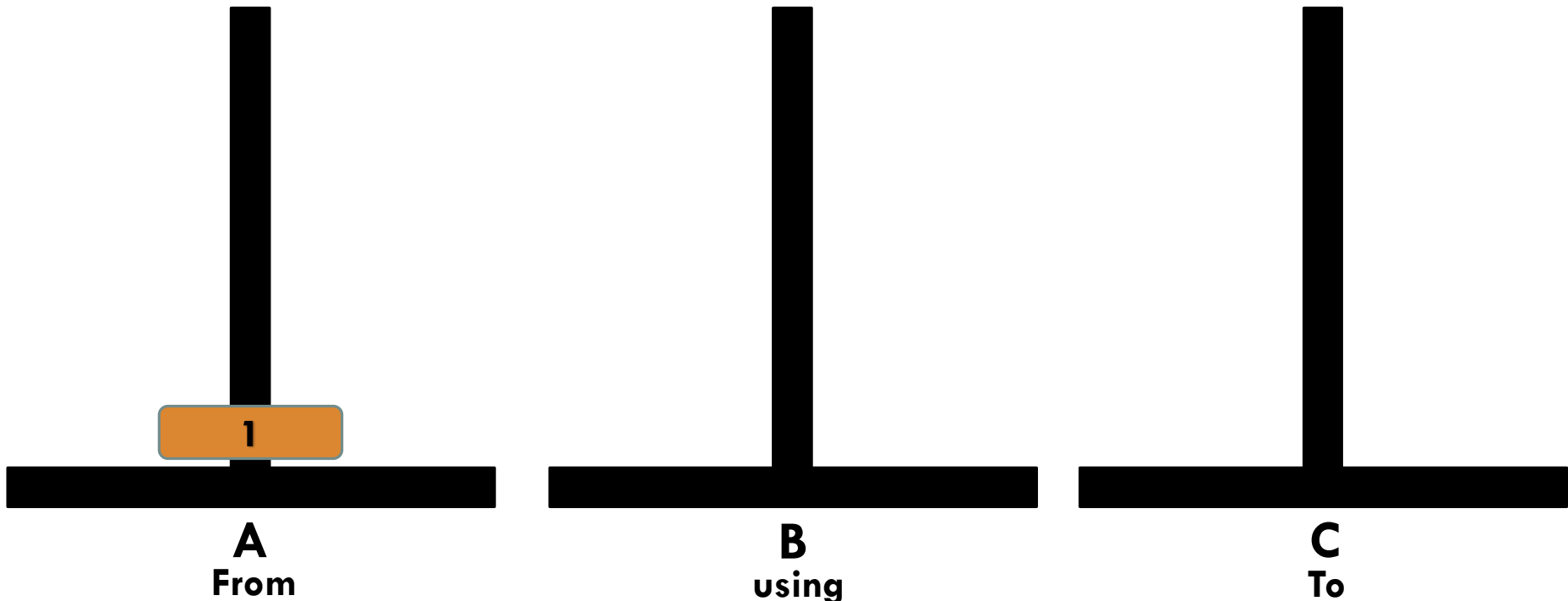
EXAMPLE 2: TOWER OF HANOI PUZZLE

Tower of Hanoi – 1 Disc



EXAMPLE 2: TOWER OF HANOI PUZZLE

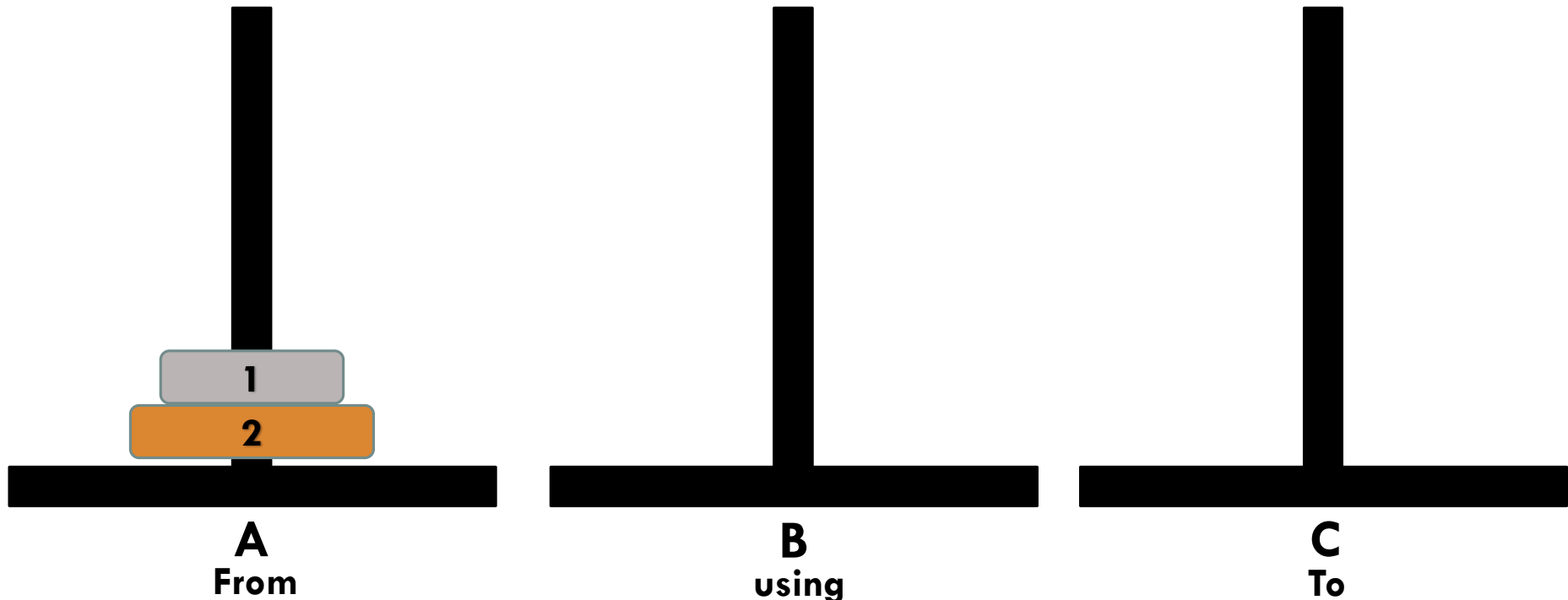
Tower of Hanoi – 1 Disc



- Moved a disc from A to C

EXAMPLE 2: TOWER OF HANOI PUZZLE

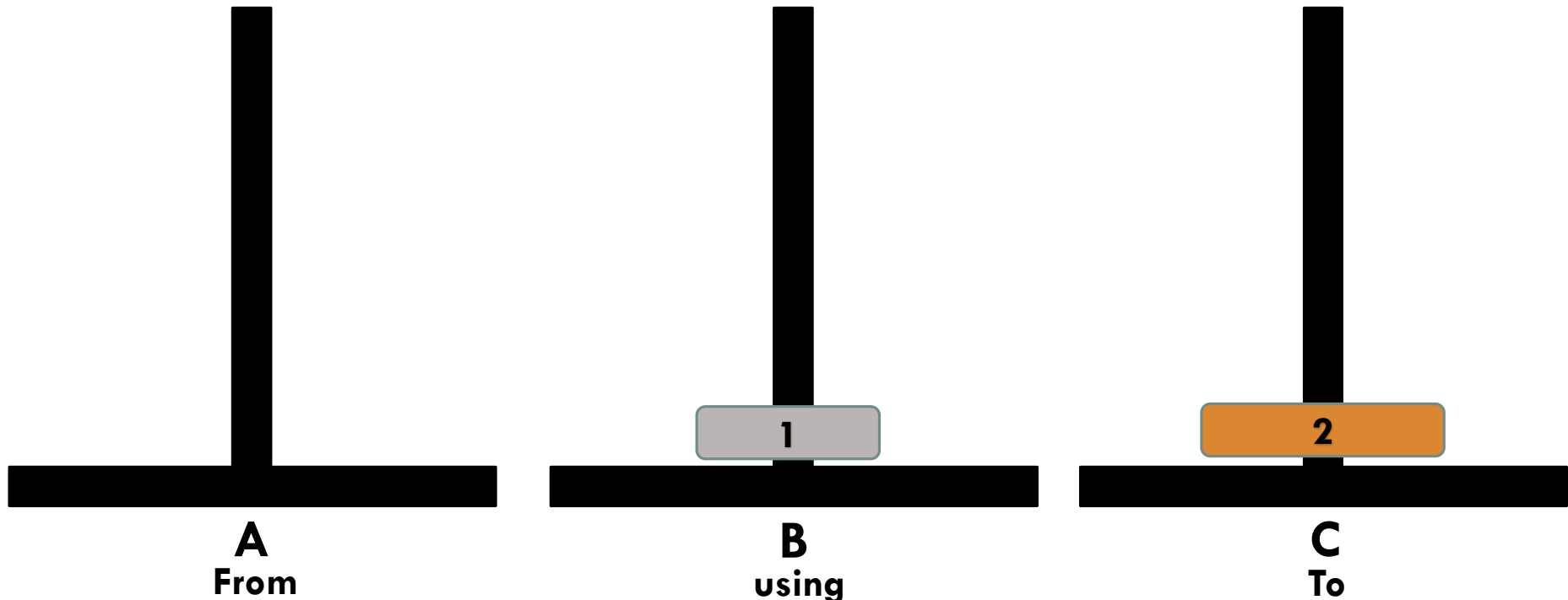
Tower of Hanoi – 2 Discs



- Moved a disc from A to B using C
- Moved a disc from A to C

EXAMPLE 2: TOWER OF HANOI PUZZLE

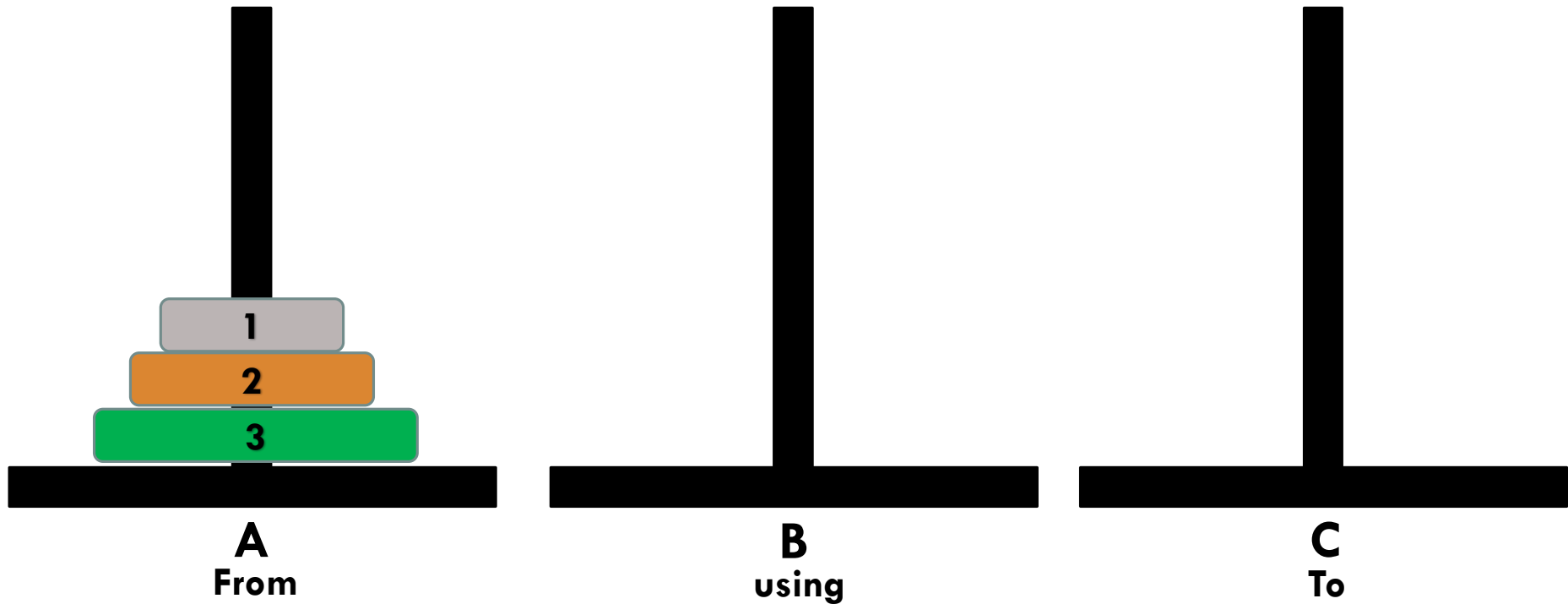
Tower of Hanoi – 2 Discs



- Moved a disc from A to B using C
- Moved a disc from A to C
- Moved a disc from B to C using A

EXAMPLE 2: TOWER OF HANOI PUZZLE

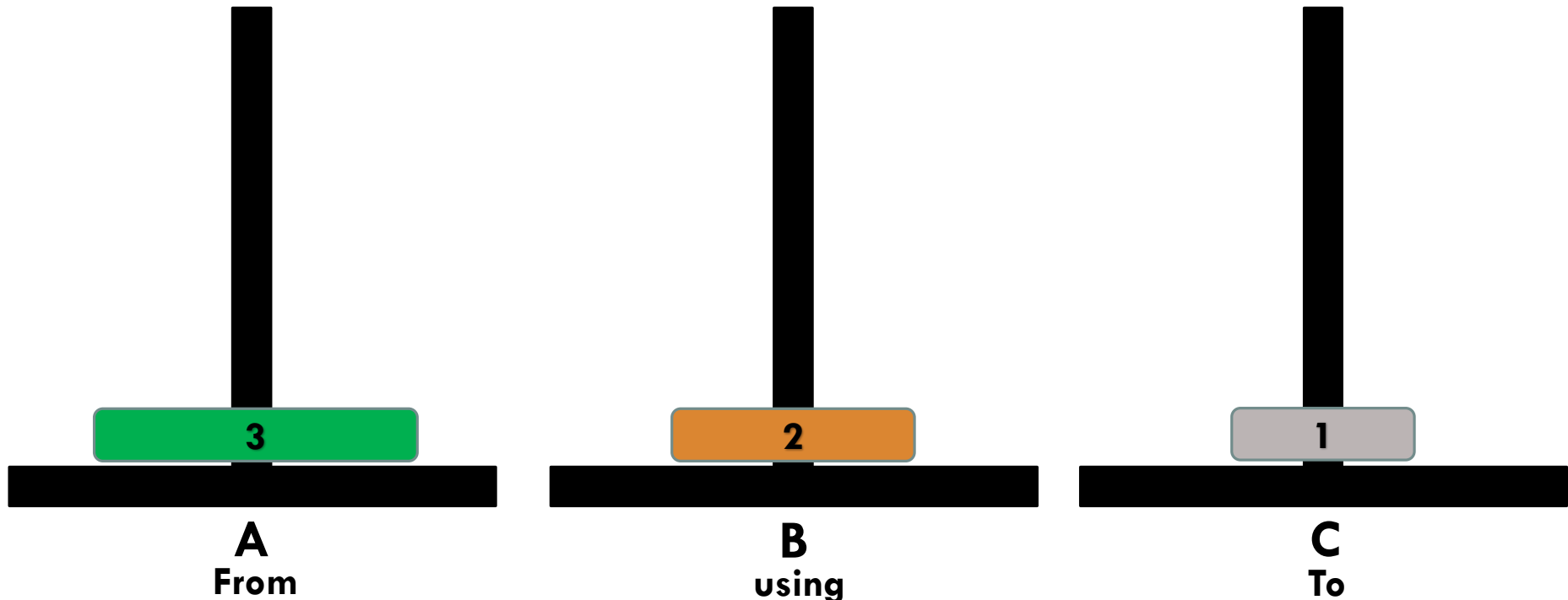
Tower of Hanoi – 3 Discs



- Moved 2 discs from A to B using C

EXAMPLE 2: TOWER OF HANOI PUZZLE

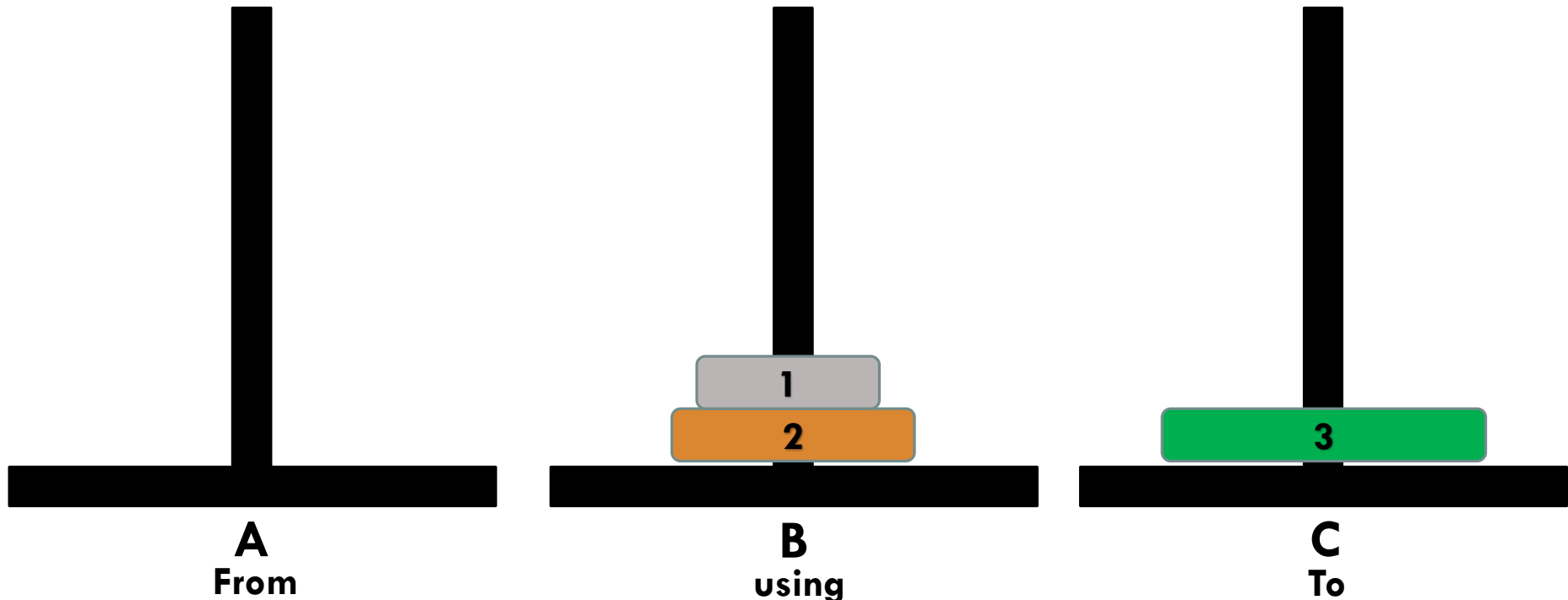
Tower of Hanoi – 3 Discs



- Moved 2 discs from A to B using C
- Moved a disc from A to C

EXAMPLE 2: TOWER OF HANOI PUZZLE

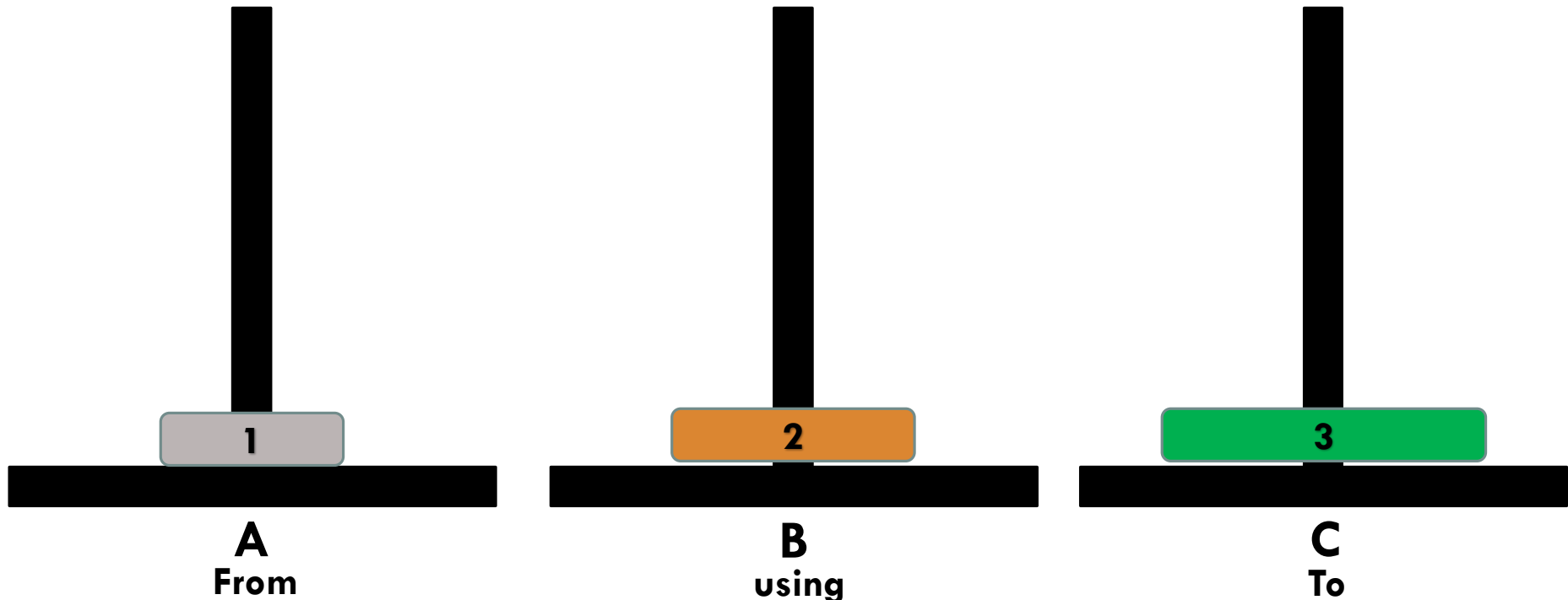
Tower of Hanoi – 3 Discs



- Moved 2 discs from A to B using C
- Moved a disc from A to C
- Moved 2 discs from B to C using A

EXAMPLE 2: TOWER OF HANOI PUZZLE

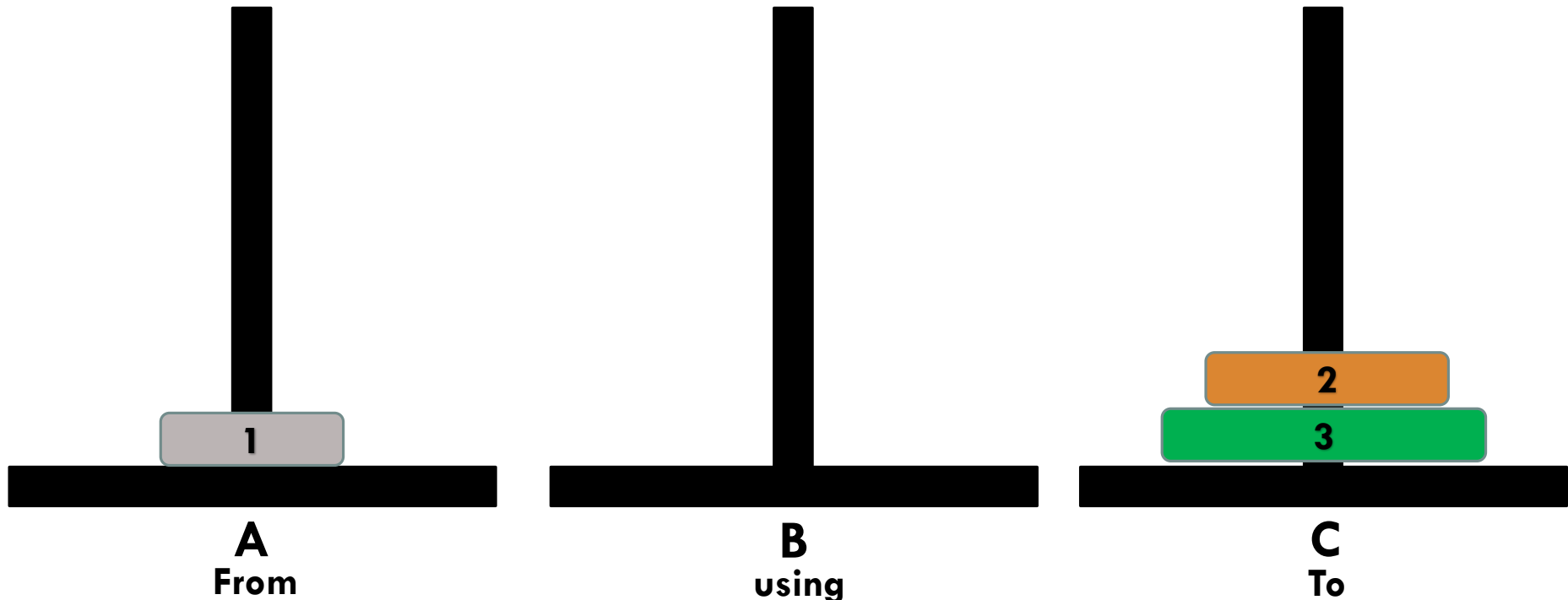
Tower of Hanoi – 3 Discs



- Moved 2 discs from A to B using C
- Moved a disc from A to C
- Moved 2 discs from B to C using A

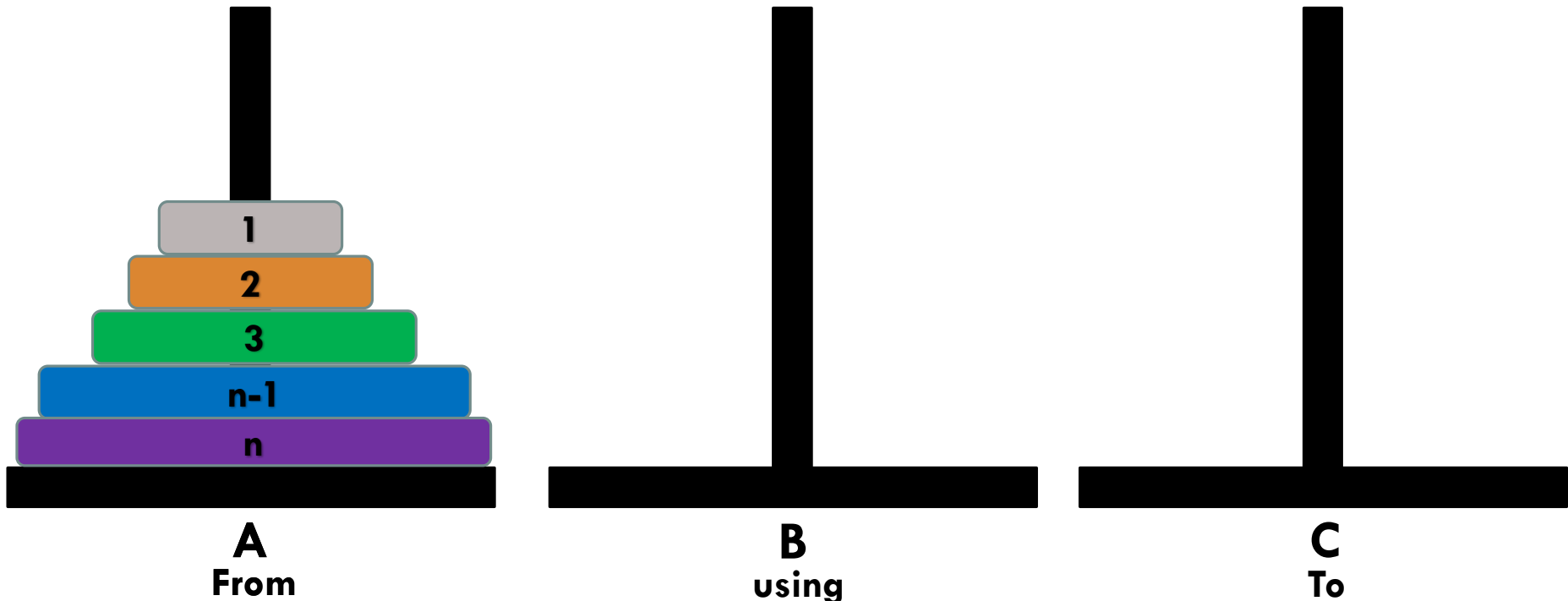
EXAMPLE 2: TOWER OF HANOI PUZZLE

Tower of Hanoi – 3 Discs



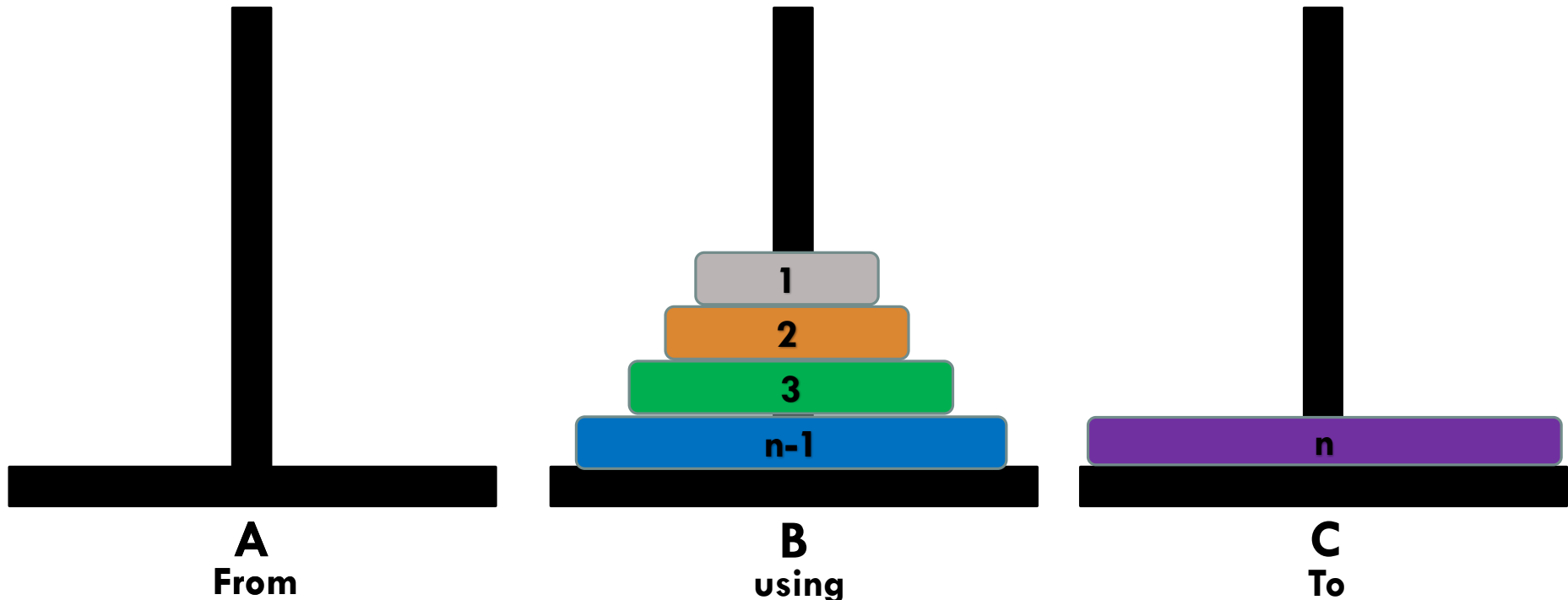
- Moved 2 discs from A to B using C
- Moved a disc from A to C
- Moved 2 discs from B to C using A

RECURSIVE PROCEDURE TO SOLVE N-TOH PROBLEM



- Moved $n-1$ discs from A to B using C
- Moved a disc from A to C

RECURSIVE PROCEDURE TO SOLVE N-TOW PROBLEM

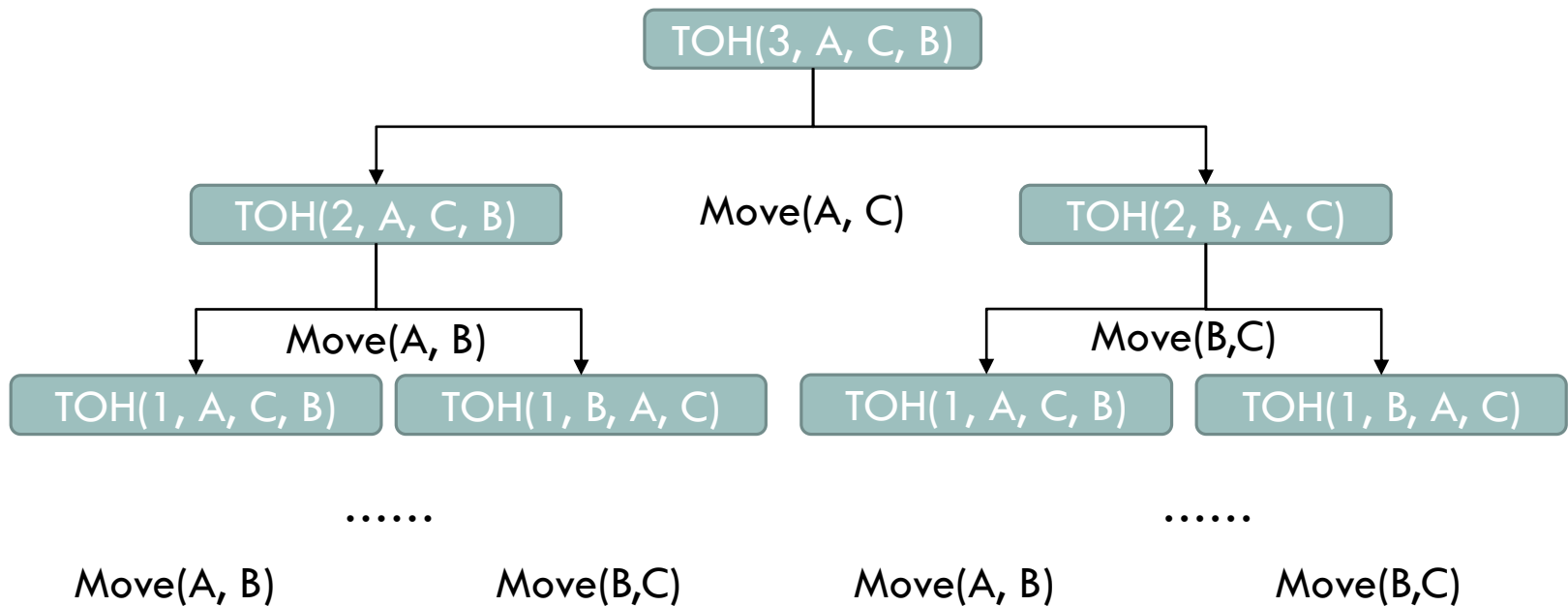


- Moved $n-1$ discs from A to B using C
- Moved a disc from A to C
- Moved $n-1$ discs from B to C using A

RECURSIVE ALGORITHM TO SOLVE TOH PROBLEM

```
TOH( n , A, B , C){  
    If( n > 0){  
        TOH(n-1, A, C, B);  
        Move a disc from A to C  
        TOH(n-1, B, A, C)  
    }  
}
```

RECURSIVE CALL TO SOLVE TOH – 3 DISCS





EFFICIENCY ANALYSIS OF TOH ALGORITHMS

$$M(n) = M(n-1) + 1 + M(n-1) \quad \text{for } n > 1, M(1) = 1.$$

$$M(n) = 2 M(n-1) + 1 \quad \text{for } n > 1, \quad M(1) = 1.$$

$$M(n) = 2 M(n-1) + 1$$

sub. $M(n-1)$ with $2M(n-2) + 1$

$$= 2[2M(n-2) + 1] + 1$$

$$= 4 M(n-2) + 2 + 1$$

$$= 2^2 M(n-2) + 2^1 + 2^0$$

sub. $M(n-2)$ with $2M(n-3) + 1$

$$= 2^2 [2M(n-3) + 1] + 2 + 1$$

$$= 2^3 M(n-3) + 2^2 + 2^1 + 2^0$$

EFFICIENCY ANALYSIS OF TOH ALGORITHMS

$$= 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \dots + 2^2 + 2^1 + 2^0$$

$$C(n) = \sum_{k=0}^{n-1} 2^k = 2^n - 1$$

$$= 2^i M(n-i) + 2^i - 1 \quad , \text{ initial condition } M(1) = M(n - (n-1)) = 1$$

$$= 2^{n-1} M(n - (n-1)) + 2^{n-1} - 1$$

$$= 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$$

$$O(2^n)$$

EXAMPLE ३: NUMBER OF BINARY DIGITS IN BINARY REPRESENTATION

```
BinRep ( n ){  
    // Input : binary number n  
    // Output : number of Binary digits in n's binary representation  
  
    If n = 1  
        return 1  
  
    Else  
        BinRep( $\left\lfloor \frac{n}{2} \right\rfloor$ ) + 1  
  
}
```

EXAMPLE ३: NUMBER OF BINARY DIGITS IN BINARY REPRESENTATION

$$A(n) = A\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1 \quad \text{for } n > 1, \quad A(1) = 0.$$

$$M(n) = M\left(\frac{n}{2^1}\right) + 1$$

$$= M\left(\frac{n}{2^2}\right) + 2$$

$$= M\left(\frac{n}{2^3}\right) + 3$$

$$= M\left(\frac{n}{2^k}\right) + k$$

$$= \frac{n}{2^k} = \frac{n}{n} = 1, \quad 2^k = n, \quad k = \log_2(n)$$

$$O(\log(n))$$

MASTER THEOREM :

Let $T(n)$ be a monotonically increasing function, that

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(1) = c, \text{ where } a \geq 1, b > 1, c > 0$$

if $f(n) \in \Theta(n^d)$, $d \geq 0$ are constants and

1) If $a < b^d$, $T(n) \in \Theta(n^d)$

2) If $a = b^d$, $T(n) \in \Theta(n^d \log_b n)$

3) If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

PITFALLS OF MASTER THEOREM :

You cannot use the Master Theorem if

- 1) $T(n)$ is not monotone, ex: $T(n) = \sin n$
- 2) $f(n)$ is not a polynomial, ex: $T(n) = 2T(\frac{n}{2}) + 2^n$
- 3) b cannot be expressed as a constant, ex: $T(n) = T(\sqrt{n})$

EXAMPLES : MASTER THEOREM

Example 1 :

- $T(n) = 4T\left(\frac{n}{2}\right) + n$

$$a = 4$$

$$b = 2$$

$$d = 1$$

$$4 > 2^1 ,$$

Case 3: $T(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2(4)}) = n^2$

EXAMPLES : MASTER THEOREM

Example 2:

- $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

$$a = 4$$

$$b = 2$$

$$d = 2$$

$$4 = 2^2,$$

$$\text{Case 2: } T(n) \in \Theta(n^d \log n) = \Theta(n^2 \log_2(n))$$

EXAMPLES : MASTER THEOREM

Example 3:

- $T(n) = 4T\left(\frac{n}{2}\right) + n^3$

$$a = 4$$

$$b = 2$$

$$d = 3$$

$$4 < 2^3 ,$$

Case 1: $T(n) \in \Theta(n^d) = \Theta(n^3)$

EXAMPLES : MASTER THEOREM

Example 4:

- $T(n) = T\left(\frac{n}{2}\right) + \frac{1}{2}n^2 + n$

$$a = 1$$

$$b = 2$$

$$d = 2$$

$$1 < 2^2,$$

Case 1: $T(n) \in \Theta(n^d) = \Theta(n^2)$

EXAMPLES : MASTER THEOREM

- $T(n) = 2^n T\left(\frac{n}{2}\right) + n^n$

The master theorem is not applicable,

$a = 2^n$ is *not a constant*